ATLAS – Plano de Testes da Detecção de Problemas Arquiteturais

Outubro de 2026

1 Escopo da avaliação experimental

O ATLAS permite investigar diferentes aspectos de um assistente de programação: detecção de problemas, correção e clareza das explicações, acessibilidade da escrita, contribuição do RAG e qualidade das edições propostas a partir da análise arquitetural. Avaliar todos esses aspectos exigiria instrumentos e critérios distintos, além de um volume de execuções e classificações incompatível com o escopo deste TCC. Por isso, este plano concentra a avaliação na **detecção de problemas arquiteturais presentes no código**.

O intuito do experimento é **verificar se o conjunto de instruções definido no prompt de comportamento da análise arquitetural é eficaz na detecção de problemas arquiteturais**, quando utilizado pelos modelos de linguagem avaliados. Para isso, as respostas do **modo de análise arquitetural** do ATLAS serão comparadas com uma referência de avaliação previamente definida para cada arquivo, considerando somente a identificação dos problemas. O prompt efetivamente enviado será preservado e mantido igual entre os modelos e as repetições, a partir das instruções definidas em `src/prompt/AtlasSystemPromptPolicyService.ts`, no código-fonte do ATLAS. O experimento não atribui nota à qualidade das explicações, à facilidade de compreensão, à acessibilidade da linguagem, ao cumprimento da estrutura de tópicos, à utilidade das recomendações ou à correção de código editado.

O **RAG permanece desativado em todas as execuções**. Os resultados descrevem a detecção a partir do arquivo submetido e do contexto estrutural que o modo disponibilizar igualmente a todos os modelos. Não se avalia a recuperação de outros arquivos nem seu efeito nas respostas. O modo de **análise rápida** também fica fora do experimento, pois utiliza outras instruções e outro formato de saída.

A comparação utiliza precisão, revocação (*recall*) e medida F_1, além da contagem de falsos positivos em cada código. Essas medidas dizem respeito somente à **identificação de problemas**, e não à qualidade global do ATLAS ou de um modelo para outras tarefas.

1.1 Questões de pesquisa sobre detecção de problemas

1. **QP1:** Como os modelos se comparam quanto à eficácia na detecção de problemas arquiteturais?
2. **QP2:** Qual modelo apresenta a menor ocorrência de falsos positivos?
3. **QP3:** O desempenho dos modelos varia conforme o tipo de problema arquitetural analisado?

4. **QP4:** Os modelos apresentam resultados consistentes entre diferentes execuções de um mesmo cenário?

A QP1 examina a eficácia na tarefa de detecção nos cenários avaliados, considerando o equilíbrio entre precisão e revocação pela medida F_1. Essa medida sintetiza acertos, apontamentos indevidos e omissões, sem representar o desempenho geral do modelo em outras tarefas. A QP2 examina a quantidade de falsos positivos nos códigos analisados; a QP3, os resultados separados por tipo de problema; e a QP4, a variação entre as repetições. Todas as questões se referem à mesma tarefa de detecção e aos cenários deste experimento.

1.2 Dois códigos por cenário de problemas

Cada cenário representa um tipo de problema arquitetural e contém **dois códigos independentes** a serem analisados. Cada código contém pelo menos dois problemas intencionais com tipo principal correspondente ao seu cenário, identificados individualmente na referência de avaliação. A construção concentra os mecanismos nesse tema para facilitar a comparação por tipo e reduzir a mistura de fatores. Assim, os cinco cenários fornecem dez códigos para a avaliação. Os problemas incluídos na referência de avaliação devem ser reconhecíveis pela entrada fornecida ao modelo, sem depender de outros arquivos.

Tabela 1: Cenários de problemas arquiteturais avaliados

Cenário	**Tipo de problema**	**Descrição do cenário**
1	Responsabilidades excessivas e baixa coesão	Um componente reúne responsabilidades distintas, acumula motivos de mudança e dificulta a compreensão de sua função principal.
2	Acoplamento a implementações concretas	Um componente depende diretamente de serviços, classes derivadas ou detalhes de implementação específicos, dificultando substituição e isolamento.
3	Encapsulamento inadequado de regras e estado do domínio	Estado ou operações de um objeto de domínio ficam expostos a alterações externas que contornam regras e invariantes do próprio objeto.
4	Dificuldade de extensão	A inclusão de uma nova variação de comportamento exige modificar vários condicionais ou pontos centrais do código.
5	Estado global e baixa testabilidade	Estado compartilhado, configuração global ou dependências implícitas do ambiente, como o relógio, dificultam o isolamento e o controle do comportamento em testes.

A seleção final dos dez códigos, dos problemas avaliados em cada um e das respectivas evidências é fixada antes das execuções. Cada problema tem um identificador próprio, de modo que uma resposta possa acertar alguns itens e omitir outros no mesmo código. A definição prévia do escopo por código permite verificar também apontamentos sem fundamento, classificados como falsos positivos. Problemas reais fora do escopo fixado são registrados separadamente e não são confundidos com erros do modelo.

1.3 Fontes e construção dos exemplos de código

Os códigos foram preparados a partir das definições e dos exemplos didáticos do *Code Smells Catalog*, disponibilizado no repositório Luzkan/smells. A fundamentação acadêmica apresentada pelo catálogo é a taxonomia de Jerzyk e Madeyski [1]. A revisão do repositório utilizada é `7f3f7dae09d487fbe68ad38e132135118155f30a`, consultada em 30 de setembro de 2026 [2]. O catálogo fornece conceitos e referências; não constitui um conjunto de programas e referências de avaliação já validado para este experimento.

Os dez arquivos independentes foram preparados em TypeScript. Os caminhos dos materiais experimentais apresentados neste plano são relativos à raiz do repositório de testes. Os códigos estão no diretório:

```
deteccao-arquitetural/codigos
```

A linguagem permite utilizar exemplos completos no ambiente do ATLAS. Algumas entradas do catálogo contêm exemplos em Python; outras apresentam apenas a definição. A adaptação conserva o mecanismo de projeto investigado e altera linguagem, domínio e detalhes necessários para tornar cada arquivo autocontido. Não se trata de reprodução literal nem de códigos provenientes de projetos reais.

O cenário 1 utiliza *Divergent Change* em relatórios de vendas e admissão de alunos. O cenário 2 utiliza *Inappropriate Static* e *Base Class depends on Subclass*. O cenário 3 utiliza *Indecent Exposure* em carteiras e reservas de estoque. O cenário 4 utiliza *Conditional Complexity* e *Combinatorial Explosion*. O cenário 5 utiliza *Global Data* e *Hidden Dependencies*. A seleção prioriza responsabilidades, dependências, encapsulamento, extensão e testabilidade observáveis em um único arquivo, compatíveis com as preocupações do modo de análise arquitetural. Os mecanismos adicionais foram construídos na adaptação dentro do cenário correspondente e documentados por inspeção; não são atribuídos automaticamente ao exemplo original do catálogo.

A Tabela 2 sintetiza os mecanismos previstos em cada arquivo e a quantidade de problemas da respectiva referência. As descrições resumem o inventário; os critérios de detecção e as evidências permanecem nas referências individuais.

A **Referência de avaliação da detecção** registra **24 problemas esperados nos dez códigos**, com pelo menos dois problemas por arquivo. Todos os problemas previamente identificados dentro do escopo são documentados com identificador, tipo, evidências e critério mínimo. Nomes alternativos e consequências do mesmo mecanismo não criam novos itens. Cada código possui um Markdown próprio no diretório:

```
deteccao-arquitetural/referencias
```

O arquivo `README.md` desse diretório reúne as regras comuns e o índice das dez referências. A correspondência com as fontes está no arquivo:

```
deteccao-arquitetural/selecao-fontes.md
```

As referências individuais registram os identificadores, os tipos, as evidências e os critérios de detecção dos problemas; a seleção das fontes documenta a revisão do catálogo e as adaptações. A revisão independente e a fixação do instrumento devem ocorrer antes da coleta definitiva.

Cada arquivo concentra problemas esperados do cenário ao qual pertence. As cinco categorias utilizadas são responsabilidades e coesão, acoplamento a implementações concretas, encapsulamento de regras e estado, dificuldade de extensão e estado global e dependências ocultas. Cada item recebe o tipo principal correspondente ao cenário. Os

Tabela 2: Códigos e mecanismos previstos nas referências de avaliação

Cenário	Código	Mecanismos esperados	Problemas
1	`C1-A`	Responsabilidades concentradas no relatório de vendas; composição de mensagens e gestão da fila reunidas; políticas de acesso e retenção na mesma classe.	3
1	`C1-B`	Políticas de admissão, armazenamento e apresentação reunidas; cobrança e controle de acesso concentrados na conta do aluno.	2
2	`C2-A`	Consulta de jornada ligada ao livro de horas concreto; serviço de folha criado diretamente pelo processamento em lote.	2
2	`C2-B`	Classe base dependente dos derivados concretos; serviço de despacho fixado em impressora térmica e protocolo ESC/POS.	2
3	`C3-A`	Saldo público; histórico devolvido como coleção interna mutável; bloqueio de gastos alterável fora da operação de liberação.	3
3	`C3-B`	Coleção de estoque pública; consulta que devolve a alocação interna modificável; limite de reserva alterável sem o setter validado.	3
4	`C4-A`	Seleção de formato repetida; transformações de linhas fixadas em despacho central.	2
4	`C4-B`	Métodos multiplicados por modalidade e região; descontos fixados nos níveis de cliente existentes.	2
5	`C5-A`	Cliente e tributação globais compartilhados; validade da cotação dependente diretamente do relógio do ambiente.	3
5	`C5-B`	Fonte e moeda resolvidas por configuração global; cache estático compartilhado entre instâncias.	2
		Total	**24**

cenários 1 a 5 reúnem, respectivamente, 5, 4, 6, 4 e 5 problemas esperados. Dentro do mesmo arquivo, os itens exigem mecanismos ou unidades afetadas distintos; consequências e nomes alternativos da mesma causa não multiplicam acertos.

As relações com SOLID e GRASP acompanham os mecanismos documentados: responsabilidade única e alta coesão em C1; inversão de dependências e baixo acoplamento em C2, com aberto/fechado e polimorfismo também em C2-B; especialista na informação e baixo acoplamento em C3; aberto/fechado e proteção contra variações em C4; baixo acoplamento em C5, com proteção contra variações do ambiente também em C5-A. Esses princípios são lentes de análise. Sua menção só demonstra detecção quando a resposta reconhece o mecanismo correspondente e apresenta evidência no código.

1.4 Preparação do arquivo submetido

Cada um dos dois códigos de um cenário é submetido **integralmente**, em uma execução própria, sem destacar os trechos problemáticos. Enviar somente esses trechos revelaria a localização dos problemas e transformaria a tarefa de detecção em descrição. O conteúdo e a ordem das linhas de cada código são idênticos para todos os modelos.

Na análise arquitetural, o ATLAS envia o código **com numeração de linhas**, no formato `<linha> | conteúdo`. Para os arquivos integrais deste experimento, a numeração corresponde às linhas originais do arquivo, a partir de 1, preservando o conteúdo e a ordem, inclusive as linhas em branco. Os prefixos são metadados de localização e não integram o código-fonte. O mesmo código numerado será fornecido a todos os modelos e em todas as repetições.

As linhas indicadas nas referências de avaliação correspondem aos números enviados ao modelo e permitem localizar as evidências. A correspondência exige identificar o elemento afetado, mas não citar uma linha exata quando a classe, o método ou o relacionamento estiverem claros. Não se enviam nomes dos problemas, referência de avaliação, documentação das fontes ou indicações de qual categoria se pretende testar.

1.5 Modelos avaliados e configurações de execução

Serão avaliados dois modelos locais e um modelo em nuvem, conforme a Tabela 3.

Tabela 3: Modelos selecionados para a avaliação da detecção

Modelo	**Execução**	**Quantização local**
M1 – gemma-4-E4B-it-GGUF	Local	Q4
M2 – Qwen3.6-27B-GGUF	Local	Q4_K_M
M3 – GPT-5.6 Luna	Nuvem	–

O GPT-5.6 Luna será utilizado como referência em nuvem para investigar o quanto os modelos locais se aproximam de um possível teto de desempenho na detecção. As diferenças na medida F_1, acompanhadas de precisão, revocação e contagem de falsos positivos, permitirão comparar os resultados obtidos com as mesmas entradas e instruções. A hipótese de que o modelo em nuvem represente esse teto será examinada na análise dos resultados.

Serão utilizadas as **configurações padrão do ATLAS para cada ambiente**: as de execução local para M1 e M2 e as de execução em nuvem para M3. A versão do

ATLAS e as configurações de partida serão mantidas ao longo da coleta. Os parâmetros efetivamente aplicados, incluindo ajustes automáticos da ferramenta e eventuais restrições do provedor em nuvem, serão registrados nos metadados das execuções.

Para M1 e M2, serão utilizados os parâmetros locais de partida apresentados na Tabela 4, definidos em `src/services/AtlasLocalModelDefaults.ts`.

Tabela 4: Parâmetros locais padrão do ATLAS para M1 e M2

Parâmetro	**Valor padrão**
Temperatura (`temperature`)	0,4
Amostragem por probabilidade acumulada (`topP`)	0,95
Limite de geração (`maxTokens`)	8192 tokens
Janela de contexto inicial (`contextWindow`)	8192 tokens
Camadas na GPU (`gpuLayers`)	0 (automático)
Threads (`threads`)	0 (automático)
Tamanho do lote (`batchSize`)	0 (automático)
Tamanho do microlote (`microBatchSize`)	0 (automático)
Flash Attention (`flashAttention`)	`auto`
Tipo do cache de chaves e valores (`kvCacheType`)	`auto`
Modo de carregamento (`loadMode`)	`auto`

Também serão mantidas as opções padrão da execução local definidas em `src/repository/AtlasConfigDefaults.ts`: ajuste automático de contexto (`dynamicContextWindow = true`), preparação da engine ao abrir o ATLAS (`prepareOnAtlasOpen = true`), resposta em streaming (`stream = true`) e timeout configurado de 30 segundos (`timeout = 30`).

Nos parâmetros `gpuLayers`, `threads`, `batchSize` e `microBatchSize`, **0 representa o modo automático (`auto`)**: o ATLAS deixa os argumentos correspondentes sem definição explícita, e a engine instalada define os valores utilizados. As opções `auto` de Flash Attention, cache e carregamento também seguem os padrões da engine. A janela de contexto poderá aumentar automaticamente quando a requisição exceder sua capacidade inicial. A janela efetiva e os demais parâmetros aplicados serão registrados por execução.

Para M3, os padrões de partida definidos em `src/repository/AtlasConfigDefaults.ts`, no código-fonte do ATLAS, são temperatura 0,4, `topP` de 0,95, limite de geração de 8192 tokens, streaming habilitado e timeout configurado de 30 segundos. Também são padrão `limitPayload = true`, `dynamicMaxTokens = false` e `sendOnlyRequiredParameters = false`. O registro dos parâmetros efetivamente enviados permitirá identificar restrições ou adaptações aplicadas pelo provedor.

Antes da coleta, serão registrados os identificadores exatos dos modelos, os provedores, a versão do ATLAS, as configurações de geração e o número de repetições. Também serão registrados data e horário de cada execução. Para os modelos locais, serão anotados o arquivo GGUF, a variante de quantização, a janela de contexto, o limite de geração, a versão do `llama.cpp`, o tipo de engine e o ambiente de execução. Diferenças impostas pelos provedores serão documentadas na interpretação.

A Tabela 5 apresenta um modelo de registro com campos que serão preenchidos antes das execuções. Os dados da engine local serão associados a M1 e M2. Caso utilizem a mesma instalação, esse uso compartilhado será registrado; caso utilizem versões ou engines diferentes, os dados de cada modelo serão documentados separadamente.

Tabela 5: Campos para registro do ambiente de execução antes da coleta

Campo	**Valor a registrar**
Versão ou commit do ATLAS	[PREENCHER: versão ou commit utilizado]
Versão do `llama.cpp`	[PREENCHER: tag da release ou commit utilizado]
Engine local	[PREENCHER: CPU, CUDA ou Vulkan]
Release da engine no GitHub	[PREENCHER: URL da release utilizada]
Pacote da engine	[PREENCHER: nome do arquivo baixado do GitHub]
Provedor em nuvem de M3	[PREENCHER: provedor utilizado]

Todos recebem o mesmo arquivo, a mesma solicitação e as mesmas instruções do modo de análise arquitetural. O RAG é desativado explicitamente e sua ausência no contexto enviado é verificada antes da coleta. Não se acrescentam trechos de outros arquivos, documentos externos ou histórico de conversa. Informações estruturais automáticas do editor, se presentes, são mantidas iguais para todos e registradas como parte da entrada.

Cada combinação de modelo e código será executada **cinco vezes**, em sessões novas. Com dez códigos, três modelos e cinco repetições, serão realizadas **150 execuções**. Cada problema esperado será avaliado cinco vezes por modelo, totalizando **15 avaliações por problema** entre os três modelos. Os problemas presentes no mesmo código serão avaliados juntos em cada execução. Respostas interrompidas por erro técnico ou limite de contexto serão registradas e tratadas conforme uma regra fixada antes da análise, sem substituição seletiva de resultados desfavoráveis.

A coleta será organizada em **cinco rodadas**. Em cada rodada, cada modelo analisará os dez arquivos uma vez, percorrendo os códigos de `C1-A` a `C5-B` na mesma ordem. A rodada seguinte corresponderá à próxima repetição de cada combinação. A ordem dos modelos será alternada entre as rodadas e registrada nos metadados. Assim, a coluna Repetição identificará uma análise do mesmo arquivo pelo mesmo modelo, e a consistência será examinada por arquivo e por problema esperado.

1.6 Definição dos problemas esperados e critérios de detecção

A unidade de avaliação é **cada problema esperado em cada código**. O cabeçalho de cada referência individual identifica o arquivo, o cenário, a quantidade de problemas esperados, a fonte do mecanismo principal e as regras comuns. Cada problema possui identificador e nome, seguidos pelos tópicos:

- **Tipo:** categoria principal correspondente ao cenário;
- **Decisão ou fragilidade observável:** mecanismo de projeto que caracteriza o problema;
- **Elementos e evidências:** classes, operações, estados ou relações afetados, com regiões verificáveis do código;
- **Critério mínimo de detecção:** reconhecimento do mecanismo e do elemento afetado que será procurado na resposta;

- **Variações de nome:** pelo menos três denominações alternativas para o mesmo problema;
- **Formulações equivalentes aceitas:** termos e interpretações que expressam o mesmo mecanismo;
- **Afirmações insuficientes:** exemplos de apontamentos que, isoladamente, não demonstram a detecção;
- **Distinção e contagem:** critérios para separar problemas independentes e reunir consequências ou repetições da mesma causa.

A referência registra **os mecanismos que devem ser reconhecidos em cada código**. Impacto, severidade, qualidade da redação e proposta de refatoração não recebem pontuação. Será considerada a resposta completa, incluindo a visão geral e os tópicos posteriores. Uma detecção pode ser expressa como tensão, fragilidade ou risco sustentado por evidência local, mesmo quando o modelo considera o desenho taticamente aceitável ou recomenda adiar uma mudança. Não se exige nome exato do problema, violação formal de princípio ou refatoração imediata. Uma descrição neutra de funções, sem reconhecer a tensão ou fragilidade, não demonstra detecção.

Para cada resposta, cada item esperado do código pode gerar no máximo um VP; se não for identificado, gera um FN. Um apontamento sem fundamento verificável nas categorias e nos elementos avaliados gera FP, mesmo quando a mesma resposta acerta outros problemas. Problemas adicionais reais fora do conjunto pré-definido são registrados sem pontuação; não são transformados em FP apenas por não constarem da referência de avaliação. Formulações equivalentes e repetições do mesmo problema são reunidas em uma única identificação, sem multiplicar acertos.

Antes da coleta, os dois códigos de cada cenário são inspecionados separadamente para confirmar que todos os itens esperados são identificáveis pela entrada fornecida. A lista de problemas e os critérios de correspondência são então fixados. Achados reais não previstos podem ser documentados à parte, mas não são acrescentados retroativamente ao denominador da revocação, evitando adaptar a referência de avaliação às respostas dos modelos.

1.7 Procedimento de avaliação da detecção

A coleta seguirá as mesmas etapas para todos os modelos:

1. serão fixados os modelos, os dois códigos de cada cenário, os problemas esperados por código, as configurações e o protocolo de cinco repetições por combinação de modelo e código;
2. será verificado se cada problema da referência de avaliação tem evidência suficiente na entrada do respectivo código;
3. será aberto o arquivo no ATLAS, será iniciada uma sessão sem histórico e será selecionado o modo de análise arquitetural;
4. o RAG será desativado e será verificado se nenhum trecho recuperado integra a entrada enviada ao modelo;

5. será enviada a mesma solicitação e serão armazenados a resposta integral, a configuração e eventuais erros;

6. serão completadas cinco rodadas, com uma análise de cada arquivo por modelo em cada rodada e alternância da ordem dos modelos;

7. a identidade do modelo nas respostas será ocultada antes da classificação;

8. os apontamentos de cada resposta serão comparados com a referência do respectivo código e classificados conforme as regras de correspondência;

9. serão preenchidos os identificadores detectados, FP e Observações na planilha; VP, FN, métricas, resumos e frequências serão calculados a partir desses registros.

Os resultados brutos serão salvos a cada execução. A checagem de que o RAG estava desativado é um **controle do experimento**, não uma avaliação da recuperação.

1.8 Classificação dos apontamentos de problemas

Como a resposta é discursiva, avaliadores identificam afirmações que atribuem um problema arquitetural ao código. Uma afirmação corresponde à referência de avaliação quando descreve o mesmo problema, identifica o elemento afetado e apresenta evidência concreta verificável no arquivo principal. Terminologia diferente é aceita quando o significado coincide.

A exigência de evidência serve apenas para confirmar a **detecção**. Não se pontuam profundidade, persuasão, estilo ou clareza da justificativa. Uma observação genérica, como dizer apenas que a arquitetura poderia melhorar, não conta como identificação. Citar um princípio sem relacioná-lo a um elemento concreto também não conta.

Cada item esperado gera no máximo um verdadeiro positivo por resposta, ainda que seja repetido. Apontamentos compostos são separados por mecanismo: um mesmo parágrafo pode reconhecer mais de um item se localizar e sustentar cada mecanismo distinto. Uma crítica genérica não recebe múltiplos acertos. As referências individuais explicitam a diferença entre problemas independentes e consequências da mesma causa. Considerando as cinco categorias definidas no plano, o código integral e as regras comuns das referências:

- **Verdadeiro positivo (VP):** identificação correta de um problema esperado do código analisado;

- **Falso positivo (FP):** apontamento de um problema sem fundamento no código, dentro das categorias e dos elementos avaliados;

- **Falso negativo (FN):** problema esperado não identificado.

Um apontamento sobre problema real fora da lista esperada não recebe VP nem FP. Se uma afirmação não puder ser verificada com a entrada controlada, será registrado o caso e a razão da dúvida, sem classificação automática como falso positivo. Observações de estilo, segurança, desempenho ou bugs funcionais sem dimensão de projeto nas cinco categorias avaliadas serão registradas como fora do escopo. Essas situações serão anotadas em Observações e apresentadas junto às contagens para tornar visível o limite da referência de avaliação. A concentração dos problemas esperados em um cenário não torna incorreto, por si só, um apontamento sobre outro tipo de problema.

Avaliadores classificam respostas sem identificação do modelo. Havendo mais de um avaliador, a classificação é independente e as divergências são resolvidas por consenso. A concordância anterior ao consenso é reportada quando calculável. Esse controle informa a confiabilidade das contagens; não cria uma segunda dimensão de qualidade textual.

1.9 Registro das execuções e das detecções

As informações serão organizadas na planilha `deteccao-arquitetural/outputs/registro-deteccao/registro-execucoes.xlsx`, com três abas: **Execuções**, **Resumo por repetição** e **Comparação dos modelos**. A aba Execuções possui **150 linhas de registro, uma por execução**. Cada linha reunirá a identificação da execução, o modelo, o código analisado, a repetição, os problemas detectados, as contagens, as métricas e as observações. As colunas dessa aba são apresentadas na Tabela 6.

Tabela 6: Colunas da planilha de registro das execuções

Coluna	**Informação registrada**
Execução	Identificador único, como `EXEC-001`, associado à resposta original.
Modelo	Identificador e nome na planilha de coleta, conforme a Tabela 3.
Código	Identificador do arquivo analisado, como `C1-A`.
Repetição	Número de 1 a 5, correspondente à execução repetida para a mesma combinação de modelo e código.
Problemas detectados	Identificadores únicos dos problemas esperados corretamente detectados, separados por ponto e vírgula.
VP	Quantidade de identificadores listados em Problemas detectados.
FP	Quantidade de apontamentos distintos sem fundamento verificável no escopo avaliado.
FN	Quantidade de problemas esperados omitidos, conforme a referência do arquivo.
Precisão	Métrica calculada automaticamente a partir de VP e FP.
Revocação	Métrica calculada automaticamente a partir de VP e FN.
F_1	Medida calculada automaticamente a partir das contagens.
Observações	Anotações relevantes sobre a execução e a classificação dos apontamentos.

A coluna **Problemas detectados** listará somente os identificadores dos itens esperados cuja detecção foi confirmada conforme a referência do arquivo. Cada identificador aparecerá uma única vez por execução, e a quantidade de identificadores corresponderá à contagem de VP. Será utilizado um travessão quando nenhum item esperado tiver sido identificado. Para uma resposta cuja classificação esteja concluída, VP + FN corresponderá à quantidade de problemas esperados na referência do arquivo.

A coluna **Observações** reunirá anotações relevantes sobre a execução e a classificação dos apontamentos, como justificativas de falsos positivos, omissões, achados reais não previstos, casos indeterminados e observações fora do escopo. Também serão anotadas interrupções ou falhas técnicas.

A planilha possui listas suspensas para Modelo, Código, Repetição e combinações dos identificadores esperados em Problemas detectados. Será preenchido FP com um inteiro não negativo, inclusive zero quando não houver apontamentos indevidos. VP será obtido

da quantidade de identificadores selecionados, e FN será calculado pela diferença entre a quantidade esperada para o código e VP. As métricas dependerão do preenchimento dos campos necessários à classificação.

Por exemplo, uma linha ilustrativa de `C1-A` poderá listar `C1-A-01; C1-A-03` em Problemas detectados, com VP = 2, FP = 1 e FN = 1; precisão, revocação e F_1 serão aproximadamente 0,67. As particularidades dessa classificação poderão ser descritas em Observações.

As respostas integrais e os metadados de coleta serão preservados em arquivos identificados pelo código da execução, incluindo data e horário, configuração utilizada, situação da execução e eventuais erros. A planilha de coleta apresentará o identificador e o nome de cada modelo. Para a classificação, serão utilizadas cópias identificadas pelo código da execução, com os nomes dos modelos e demais metadados de identificação ocultados, e a correspondência entre execuções e modelos será mantida separada dos materiais apresentados aos avaliadores. Interrupções e falhas técnicas serão tratadas conforme a regra fixada antes da análise.

Na aba **Resumo por repetição**, uma rodada será considerada completa quando os dez códigos distintos tiverem classificação válida para o mesmo modelo e número de repetição, sem registros duplicados. Serão somados VP, FP e FN antes do cálculo das métricas da rodada. Na aba **Comparação dos modelos**, serão apresentadas as médias das métricas entre as rodadas completas e o desvio padrão amostral de F_1 por modelo, calculado somente com pelo menos duas rodadas completas. Os identificadores registrados permitirão recuperar o tipo de cada problema para a comparação da QP3. Os cálculos seguirão as fórmulas e os casos de denominador zero definidos na Seção 1.10.

Frequência de detecção por problema. Na aba **Comparação dos modelos**, também será apresentada uma tabela com **72 linhas**, uma para cada combinação dos 24 problemas esperados com os três modelos. Serão utilizadas as colunas **Modelo**, **Problema**, **Tipo**, **Detecções**, **Execuções avaliadas** e **Frequência**. O tipo principal será obtido da referência individual de cada problema.

As contagens serão calculadas automaticamente a partir da aba Execuções. Para cada problema, serão contadas as execuções avaliadas do arquivo correspondente nas quais seu identificador aparecerá em Problemas detectados. A frequência será a razão entre essa contagem e a quantidade de execuções avaliadas para aquele modelo e arquivo, apresentada em percentual. Somente serão consideradas repetições de 1 a 5 com classificação válida e FP preenchido. Registros duplicados para a mesma combinação de modelo, código e repetição serão excluídos dessa contagem até sua correção.

Sem execuções avaliadas, a frequência ficará em branco. Após cinco avaliações, 5/5 indicará detecção em todas, 0/5 indicará omissão em todas e valores intermediários indicarão variação entre execuções. A tabela apoiará a comparação por tipo da QP3 e a análise da regularidade das detecções da QP4. Essa análise complementará o F_1 agregado, pois execuções com a mesma pontuação poderão identificar problemas diferentes.

1.10 Métricas de detecção

A precisão mede a proporção de acertos entre os apontamentos classificados como VP ou FP, conforme a referência fixada:

$$\text{Precisão} = \frac{VP}{VP + FP}. \tag{1}$$

A revocação mede a proporção dos problemas esperados identificados:

$$\text{Revocação} = \frac{VP}{VP + FN}. \tag{2}$$

A medida F_1 sintetiza precisão e revocação pela média harmônica. Na planilha, será utilizada sua forma equivalente em contagens:

$$F_1 = 2\,\frac{\text{Precisão} \times \text{Revocação}}{\text{Precisão} + \text{Revocação}} = \frac{2VP}{2VP + FP + FN}. \tag{3}$$

As contagens de VP, FP e FN são somadas por modelo em cada rodada completa dos dez códigos antes do cálculo das métricas gerais. Cada rodada contém 24 problemas esperados. O denominador por arquivo corresponde à quantidade registrada em sua referência individual. Para a QP3, os itens são agrupados pelo seu tipo principal, correspondente ao cenário do arquivo. Também se apresentam as contagens e a revocação por código e por cenário, com o número de itens esperados em cada grupo. Os falsos positivos são informados por código e no total. As repetições permitem relatar média, desvio padrão e frequência de identificação de cada problema.

Quando $VP + FP = 0$, a precisão é indefinida. Como cada código contém pelo menos dois problemas esperados, a revocação tem denominador definido. Se nenhum problema esperado for identificado, $F_1 = 0$, inclusive quando não houver apontamentos; a forma em contagens permite calcular esse caso. Valores de precisão indefinida serão identificados na planilha e excluídos apenas da média de precisão. Se não houver valor definido dessa métrica entre as rodadas completas, a média também será indicada como indefinida. Rodadas incompletas serão excluídas das médias.

A acurácia clássica não é usada porque a tarefa não oferece um conjunto fechado e comparável de verdadeiros negativos em texto livre. Precisão e revocação são apresentadas juntas: um modelo pode detectar muitos problemas e gerar ruído, ou apontar poucos problemas com maior seletividade.

1.11 Apresentação dos resultados de detecção

Os resultados são apresentados por modelo, com VP, FP, FN, precisão, revocação, F_1, falsos positivos por código e variação entre execuções. O desempenho por tipo de problema mostra em quais itens a detecção foi mais ou menos frequente. Exemplos de acertos, omissões e apontamentos indevidos podem ilustrar as contagens, sem avaliação da qualidade textual.

Um F_1 maior indica melhor equilíbrio entre acertos e apontamentos indevidos **nos códigos avaliados**. A conclusão considera também precisão, revocação e falsos positivos por código. O estudo é comparativo e exploratório; não estabelece superioridade universal de um modelo para qualquer projeto ou tarefa.

1.12 Limites e ameaças à validade da detecção

Escopo deliberadamente restrito. O plano testa somente detecção de problemas arquiteturais no modo de análise arquitetural, sem RAG. Não mede clareza ou correção da explicação além da evidência mínima que confirma uma identificação, nem acessibilidade da escrita, qualidade de edições ou contribuição do contexto recuperado. Detecção favorável não demonstra que uma resposta seja compreensível ou que uma solução proposta esteja correta.

Cenários construídos e amostra limitada. Problemas preparados podem ser mais explícitos que em projetos reais. Dois códigos por cenário limitam a generalização e podem diferir em dificuldade. A referência ao catálogo sustenta a escolha dos conceitos, mas não valida automaticamente as adaptações ou suas referências de avaliação. Os arquivos contêm quantidades diferentes de problemas esperados e, portanto, têm pesos distintos nas métricas agregadas. Apresentam-se resultados por arquivo, por cenário e por tipo, com seus denominadores. As conclusões se restringem aos arquivos, modelos e configurações avaliados.

Familiaridade com material público. As definições e os exemplos da fonte podem ter integrado os dados de treinamento dos modelos. A adaptação de linguagem e domínio não elimina esse risco. Não se infere capacidade de detectar problemas inéditos ou em grandes sistemas a partir desta amostra.

Ausência de códigos sem problemas. Todos os códigos contêm problemas previstos na referência de avaliação. É possível contar apontamentos incorretos neles, mas o experimento não mede diretamente como os modelos se comportam diante de códigos inteiramente adequados.

Contexto disponível sem RAG. Só entram na referência de avaliação problemas identificáveis a partir da entrada controlada. Problemas dependentes de evidências presentes apenas em outros arquivos ficam fora desta avaliação.

Julgamento humano. Respostas discursivas admitem interpretações diferentes. Referência de avaliação prévia, regras de correspondência, cegamento e classificação independente reduzem esse risco, mas não o eliminam.

Variação entre execuções e provedores. Modelos podem produzir saídas diferentes para a mesma entrada, e serviços em nuvem podem mudar. Repetições e registro de versões, datas e parâmetros tornam a limitação visível, sem garantir reprodução exata.

Respostas interrompidas. Falhas técnicas ou truncamentos podem afetar desigualmente os modelos. A regra de tratamento é definida antes da coleta e o número de ocorrências é reportado por modelo.

Referências

[1] JERZYK, Marcel; MADEYSKI, Lech. Code Smells: A Comprehensive Online Catalog and Taxonomy. In: *Developments in Information and Knowledge Management Systems for Business Applications*. Studies in Systems, Decision and Control, v. 462. Cham: Springer, 2023. p. 543–576. DOI: 10.1007/978-3-031-25695-0_24.

[2] JERZYK, Marcel. *Code Smells Catalog*. Repositório Luzkan/smells, revisão de referência indicada na Seção 1.3. Disponível em: `https://github.com/Luzkan/smells`. Catálogo: `https://www.codesmells.org/`. Acesso em: 30 set. 2026.